How Flink Handles Stateful Streaming at Large Scale

From Kafka Message to Exactly-Once Delivery

Xuan-Cuong Le

cuongkane.com

April 8, 2026

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

7 Chapter 5–6: Enrich & Sink

What Is Stateful Stream Processing?

Most stream processing is **stateless**: message in, transform, push out.

But many real-world problems require **remembering things across messages**:

- **Deduplication** – "Have I seen this event ID before?"
- **Windowed aggregations** – "How many orders in the last 5 minutes?"
- **Pattern detection** – "Alert if a user fails login 3 times in a row"

The moment your processor needs to remember something between messages, you have **stateful** stream processing.

The Naive Approach: External State Store

A Kafka consumer that checks PostgreSQL/Redis for every message.

This works at moderate scale. For thousands – even tens of thousands – of messages/second, an external store is perfectly fine.

The question is what happens when you outgrow it.

Why External Stores Break Down at Scale

- **Network round-trips kill throughput**
  - At 1M msg/s across 8 partitions: 2M round-trips/second to Redis
  - Each round-trip adds 0.5–2ms latency
  - Pipeline bottleneck becomes the network, not the computation
- **Consistency is painful**
  - Crash between Redis write and Kafka produce → inconsistent state
  - Exactly-once across two systems requires distributed transactions
- **Scaling state independently from compute is wasteful**
  - Redis cluster scales on state size, consumers on throughput
  - Coupled in practice, managed separately

External state turns **every message** into a distributed system problem.

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

7 Chapter 5–6: Enrich & Sink

Flink's Approach: Co-located State

State lives inside the stream processor.

- Each parallel operator instance has its own **local RocksDB** store
- No network round-trips for state access
- Local disk reads that hit an in-memory cache 95%+ of the time

But co-located state introduces new problems:

- How do you make local state **durable**?
- How do you **recover** it after a crash?
- How do you **redistribute** it when you scale up?

Flink's answer: checkpoint barriers, incremental snapshots, and two-phase commits.

The Pipeline We'll Trace

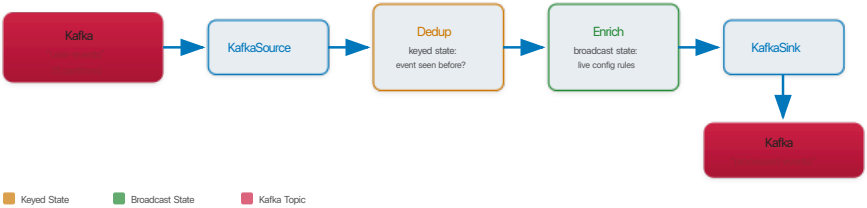


Figure 1: A single Kafka message through: Source → Dedup → Enrich → Sink

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

7 Chapter 5–6: Enrich & Sink

Flink's Moving Parts

JobManager – The brain

- Coordinates the entire job
- Triggers checkpoints, handles failures
- Contains: Dispatcher, ResourceManager, JobMaster

TaskManager – The workers

- Separate JVM process (one K8s Pod)
- Registers with JobManager
- Offers computational capacity

Slot – Resource container

- Logical slice of TaskManager resources
- Hard-partitioned memory budget

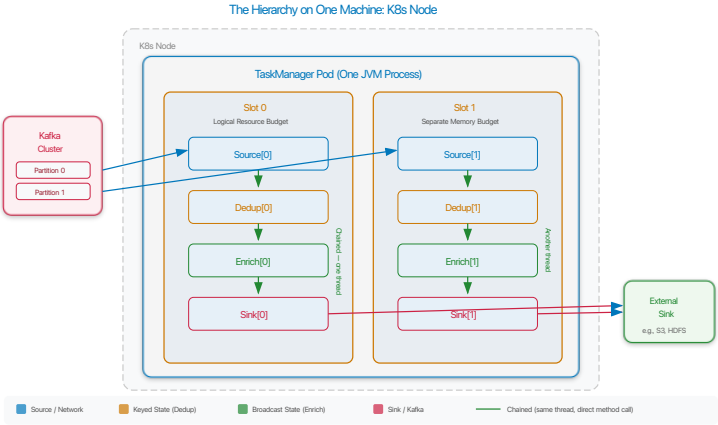
Operator – Processing step

- KafkaSource, Dedup, Enrich, KafkaSink
- Defines *what* to do with data

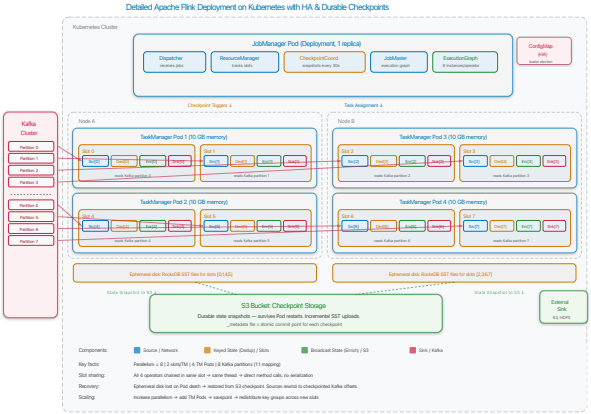
Subtask – Parallel instance

- Dedup with parallelism=8 $\rightarrow$ 8 subtasks

Hierarchy on One Machine



Flink on Kubernetes (parallelism=8, 2 slots/TM)



Key Observations

- **Each TaskManager Pod is one JVM process**
  - 2 slots share the JVM heap but each gets **isolated managed memory**
- **8 Kafka partitions, parallelism 8, 8 slots** – ideal 1:1 mapping
  - Flink never splits one Kafka partition across multiple readers
- **Slot sharing**: all 4 operators run in the **same slot** for the same subtask index
  - Chained into a single Java thread; data flows via direct method calls
- **RocksDB state on ephemeral disk** – disappears when Pod dies
  - Checkpoint data on S3 (durable); new Pods rebuild from S3
- **Scaling up** = more TM Pods + savepoint to redistribute state

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

7 Chapter 5–6: Enrich & Sink

Chapter 1: A Message Is Born

A user event arrives in Kafka **partition 3, offset 50701**:

```
{
  "event_id": "evt-88821",
  "user_id": "user-42",
  "action": "purchase",
  "amount": 59.99,
  "timestamp": 1712100000000
}
```

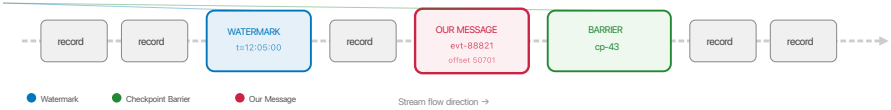
Chapter 2: Flink Picks It Up

Source[3] – the 4th parallel instance of KafkaSource, running in **Slot 3, TaskManager Pod 2, Node A**.

1. Background fetcher thread calls Kafka's `poll()`
2. Message arrives in a batch; placed into a hand-over queue
3. Slot's task thread picks it up
4. Source deserializes raw bytes into a Flink `StreamRecord`

Flink does **NOT** use Kafka consumer group offsets for recovery. It stores offsets in its own checkpoint state.

Stream Markers: Watermarks & Checkpoint Barriers



Watermarks – "When Can We Close a Time Window?"

- A timestamp that flows through the stream meaning: **"all events before this value have already arrived"**
- Each **source subtask** generates watermarks based on event timestamps
- Each **subtask** tracks watermarks from all input channels
- Takes the **minimum** across all inputs

One straggling Kafka partition holds back **all** downstream time-based operations.

Checkpoint Barriers – "Take a Snapshot at This Exact Point"

- **JobManager's CheckpointCoordinator** sends "start checkpoint 44" to all sources
- Each **source subtask** injects a barrier into its output stream
- Barrier divides the stream: before → checkpoint 44, after → checkpoint 45
- Each **subtask** handles barrier alignment independently

Aligned mode (default)

- Blocks fast channels
- Processes only slow ones

Unaligned mode (opt-in)

- Continues all channels
- Includes in-flight records in checkpoint
- Avoids backpressure; larger checkpoints

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

7 Chapter 5–6: Enrich & Sink

Chapter 3: Does This Message Need a Network Hop?

Dedup is keyed by `event_id`. After `keyBy()`, Flink routes each record to the subtask that **owns** that key's state.

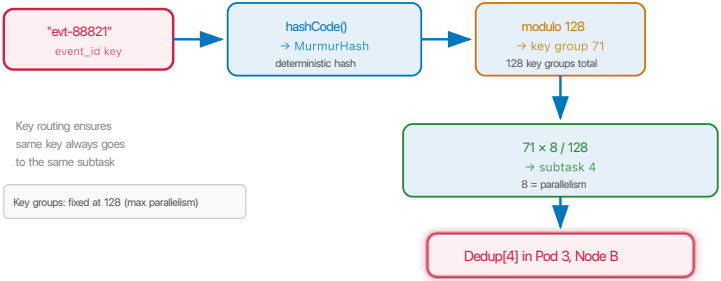


Figure 2: Key hashing: event_id → hash → key group → subtask

What Happens on the Wire

Source[3] in Pod 2 (Node A) → **Dedup[4] in Pod 3 (Node B)**

1. **Serialization**: message → bytes in a 32 KB network buffer
2. **Partitioning**: `ResultPartition` has 8 sub-partitions; ours goes to sub-partition 4
3. **TCP transport**: Netty over TCP, through K8s network (CNI plugin)
4. **Deserialization**: Dedup[4]'s input gate → `StreamRecord` → Dedup operator

Backpressure: If Dedup[4] is slow → input buffers fill → no credits sent → sender blocks → Source[3] slows Kafka `poll()` rate. **No data is ever dropped.**

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

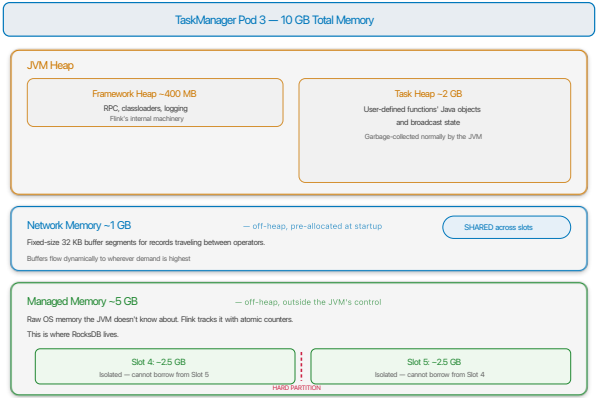
4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

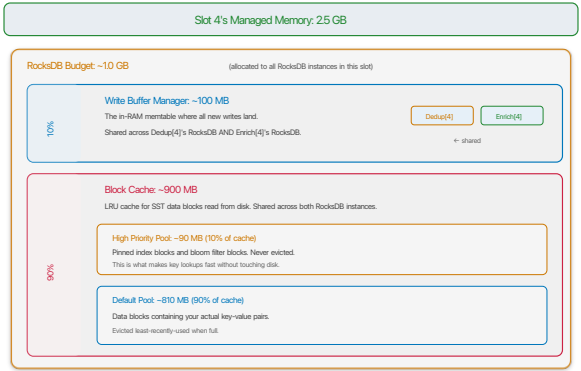
7 Chapter 5–6: Enrich & Sink

TaskManager Memory Layout



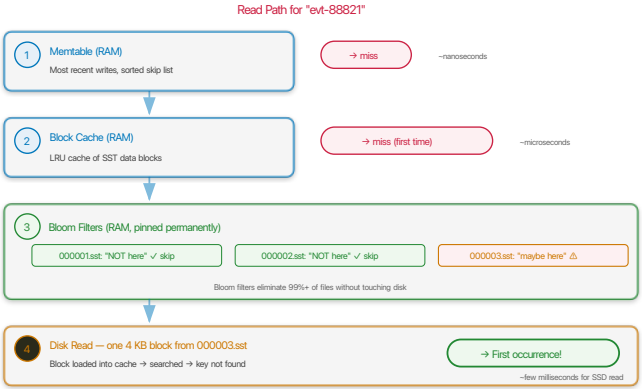
Managed memory is **hard-partitioned per slot**. Slot 4 cannot steal from Slot 5.

Slot Memory Breakdown



Dedup[4] and Enrich[4] are separate RocksDB instances but **share** the block cache within Slot 4. They never compete with Slot 5's RocksDB.

RocksDB Read Path – "Have I Seen This Before?"



RocksDB Read Path – Step by Step

1. **Memtable (RAM)** – most recent writes. Cost: **nanoseconds**
2. **Block cache (RAM)** – cached 4 KB blocks from SST files. Cost: **microseconds**
3. **Bloom filters (RAM)** – eliminates 99%+ of SST files without disk I/O
4. **Disk read (SSD)** – read specific 4 KB block. Cost: **few milliseconds**

For dedup with short time windows, **block cache hit rate is 95%+**. Most lookups are RAM-speed.

RocksDB Write Path – Marking "evt-88821" as Seen

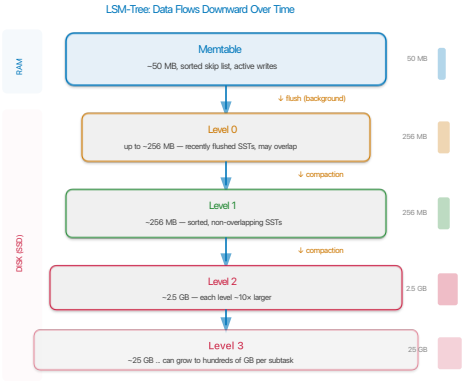
Your processing thread never waits on disk:

1. Serialize key + value into a **write batch**
2. Flush batch into the **memtable** (a skip list in RAM). Cost: **microseconds**
3. Your thread returns immediately. Everything below is background.

Background flow (RAM → Disk):

1. Memtable fills up → marked **immutable**; new empty memtable created
2. Flush thread writes immutable memtable as **SST file** at Level 0
3. Compaction threads merge L0 → L1 → L2 → . . . (each level ∼10x larger)

LSM-Tree: From Memtable to Disk



SST Files on Disk

RocksDB Directory Structure

```
/tmp/flink/job_{jobId}_op_dedup_uuid_{random}/db/
├── 000001.sst        ← Level 1 SST, ~64 MB, sorted, immutable
├── 000002.sst        ← Level 1 SST
├── 000003.sst        ← Level 2 SST (produced by compaction)
├── 000004.sst        ← Level 0 SST (recently flushed from memtable)
├── MANIFEST-000005          ← Metadata: which SSTs exist at which level
├── CURRENT                  ← Points to the current MANIFEST
└── OPTIONS-000006           ← RocksDB configuration snapshot
```

Data files (SST) · Recently flushed · Metadata · Configuration · SSTs are immutable once written

- Each Flink state descriptor → separate **column family** in RocksDB
- SST files are **immutable** – never modified after creation
- Snapshots = **hard links** to current SST files (microseconds, any state size)

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

7 Chapter 5–6: Enrich & Sink

Chapter 5: The Enrich – Broadcast State

Dedup[4] → Enrich[4]: **same slot, same thread, direct method call** (nanoseconds).

How Broadcast State Works

- Keyed state is *partitioned*
- Broadcast state is **replicated**
- Every subtask holds the same complete copy
- Lives as a **Java HashMap in JVM heap**

Write Restriction

- Processing user events: **read-only** access
- Processing config updates: **write** access
- Why? User events are partitioned; allowing writes would cause 8 copies to diverge

Chapter 6: The Sink – Writing to Kafka

Enrich[4] → Sink[4]: also **chained** – same slot, same thread.

- Each Sink subtask has its own **Kafka producer**
- All 8 Sinks write in parallel; Kafka handles concurrent appends natively
- JobManager is **not involved** in data delivery – only checkpoints and recovery

The Kafka transaction is still open. Records are physically stored but **invisible** to `read_committed` consumers. Our message is in limbo – written but not committed.

It stays this way until the next checkpoint completes.

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

7 Chapter 5–6: Enrich & Sink

Chapter 7: The Checkpoint – Making It All Durable

Every 30s (configurable), the CheckpointCoordinator triggers a distributed snapshot.

Sink Pre-Commit (when barrier reaches Sink[4]):

1. **Flush** all pending records to Kafka broker (transaction still open)
2. **Snapshot** producer ID and transaction state to S3

Finalization:

1. All 32 subtasks (8 × 4 operators) send `acknowledgeCheckpoint(44)` to JobManager
2. JobManager writes `_metadata` file to S3 – the **atomic commit point**
3. JobManager sends `notifyCheckpointComplete(44)` to Sinks
4. Each Sink calls `commitTransaction()` – records **now visible**

Exactly-Once Achieved

Our message is now fully committed:

- Kafka offset is part of the checkpoint state
- Dedup "seen" flag is in the RocksDB snapshot on S3
- Output record is visible in the output Kafka topic

If the entire cluster crashes, Flink restores from S3 and resumes from exactly this point. **No reprocessing. No duplicates.**

When Things Go Wrong – Pod Crash Recovery

TaskManager Pod 3 on Node B gets **OOM-killed**. Slot 4 & 5 – including our Dedup[4] – are gone.

1. JobManager detects failure via **heartbeat timeout**
2. K8s schedules a **replacement Pod** (possibly different node)
3. New TaskManager registers, offers 2 fresh slots
4. JobManager **restores from checkpoint 44**: downloads SST files from S3
5. KafkaSources **rewind** to checkpointed offsets
6. KafkaSinks **abort** uncommitted transactions (zombie fencing)
7. Processing resumes – **brief pause, no data lost, no duplicates**

K8s handles Pod lifecycle. Flink handles data consistency. Together, the crash is **invisible**.

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

7 Chapter 5–6: Enrich & Sink

Key Takeaways from Flink's Architecture

- **Co-locate state with computation**
  - Eliminating the network hop is the single biggest performance win
- **Immutable data structures enable cheap snapshots**
  - SST files never modified $\rightarrow$ snapshot = hard links (microseconds)
- **Barriers over pauses**
  - Chandy-Lamport algorithm: coordinate without stopping processing
- **Incremental over full**
  - 500 GB state $\neq$ 500 GB uploaded per checkpoint; only the delta
- **Two-phase commit for end-to-end exactly-once**
  - Sink pre-commits during checkpoint; visible only after finalization

1 Stateful Stream Processing

2 How Flink Solves This

3 Flink Architecture

4 Chapter 1–2: Message Ingestion

5 Chapter 3: The Dedup – Network Shuffle

6 Chapter 4: State – RAM or Disk?

7 Chapter 5–6: Enrich & Sink

Should You Actually Use Flink?

The operational commitment is serious:

- **Cluster management**: JobManagers, TaskManagers, S3, monitoring
- **Tuning**: RocksDB memory, network buffers, checkpoint intervals, parallelism
- **Upgrades**: Savepoint management, state migration
- **Debugging**: Is it compaction? GC? Network? Key skew?

Use Flink when you genuinely need it:
Millions of msg/s, terabytes of state, exactly-once, event-time processing.

Under 10K msg/s? A simple consumer with Redis works fine.
Moderate scale + exactly-once? Consider Kafka Streams.

Thank You